

Lab 2: Baseband QAM Modem

Part 2: Pulse Shaping and Matched Filtering

Summary

In Part 1 of Lab 2, you implemented the digital modulation and detection parts of a baseband QAM modem. This part of the lab will deal with pulse shaping in digital communication systems. You will implement the pulse shaping and matched filtering blocks of the baseband QAM modem shown in Figure 1.

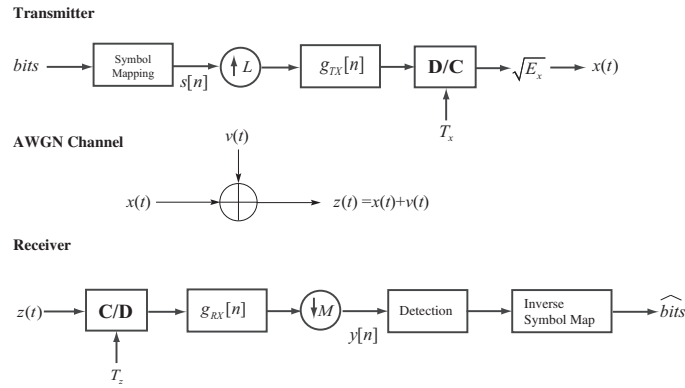


Figure 1: The system under consideration in this lab. The transmitter uses digital pulse-shaping and upsampling to create the transmit waveform. The baseband signal encounters additive white Gaussian noise at the receiver. The receiver uses digital pulse-shaping and downsampling, followed by detection, to find a good guess of the transmitted symbols. Symbol synchronization could be included as well but is not illustrated in the figure.

We begin by discussing the motivation and mathematical model for pulse shaping. You will then build the pulse shaping, up-sampling, and matched filtering blocks of the modem in LabVIEW. After verifying the correctness of your design using a simulator, you will implement your code in lab using the NI-USRP to see how your code works over a real wireless link. The goal of this part of Lab 2 is to understand the motivation for pulse shaping and observe the properties of various pulse shaping filters.

In this lab, you will have to turn in two VIs (*pulse_shaping.vi* and *matched_filtering.vi*), and answer the questions given in Section 2, as part of the pre-lab submission. The lab reports for Parts 1 and 2 of Lab 2 have to be submitted as a single lab report at the end of this lab.

1 Pulse Shaping and Matched Filtering

The objective of the course is to construct the digital signal processing blocks necessary to operate a wireless digital communication link. A digital signal (from the digital source, constructed in Part 1 of Lab 1) can not be transmitted over a communication channel. The digital signal needs to be mapped to an analog waveform that can easily be transmitted over the channel.

The model for transmission and reception considered in Lab 2 is illustrated in Figure 1. The transmitter creates a complex pulse amplitude modulated signal of the following form:

$$x(t) = \sqrt{E_x} \sum_{n=-\infty}^{\infty} s[n]g_{\text{tx}}(t - nT). \quad (1)$$

Conceptually, the mapping of bits to waveforms can be viewed in two steps.

1. **Source bits to symbols**—This involves mapping the bits emitted by the (digital) source to (possibly complex) symbols, s on a constellation, $\mathcal{C}$ determined by the modulation scheme. This was accomplished in Part 1 of Lab 2.
2. **Symbols to pulses**—This involves creating an analog pulse train to be transmitted over the wireless channel, using the complex symbols generated in the first step. This is done using a pulse-shaping filter $g_{\text{tx}}(t)$ at the transmit side.

The receiver processing depends on the type of distortion in the communication channel. Assuming an additive white Gaussian noise (AWGN) channel, the receiver performs analogous operations as at the transmitter.

1. **Pulses to symbols**—In this step, the symbols are recovered from the received pulses using a matched filter at the receive side.
2. **Symbols to bits**—This involves mapping the symbols received to bits (depending on the modulation scheme chosen), using some form of a detection technique. This was accomplished in Part 1 of Lab 2 using the maximum likelihood detection algorithm.

To implement the maximum likelihood detector as described in Part 1 of Lab 2 using single-shot detection, the pulse shaping filters $g_{\text{tx}}(t)$ and $g_{\text{rx}}(t)$ have to satisfy some properties.

- The transmit pulse shape is normalized such that $\int |g_{\text{tx}}(t)|^2 dt = 1$. This is primarily for convenience in our notation to avoid additional scaling factors.
- The receive filter $g_{\text{rx}}(t) = g_{\text{tx}}^*(-t)$, which simplifies to $g_{\text{rx}}(t) = g_{\text{tx}}(-t)$ since the pulse-shaping function is usually real. This means that the receive filter is a matched filter. Effectively the filtering by $g_{\text{rx}}(t)$ becomes a correlation with $g_{\text{tx}}(t)$ thanks to the relationship between convolution and correlation. The choice of a matched filter maximizes the received signal-to-noise ratio and thus gives the maximum likelihood detector its best performance.
- The composite filter $g(t) = \int g_{\text{rx}}(\tau)g_{\text{tx}}(t - \tau)dt$ is a Nyquist pulse shape. This means that it satisfies $g(nT) = \delta[n]$. The Nyquist pulse-shape property eliminates intersymbol interference, allowing the detector to take a very simple form as described in Part 1 of Lab 2.

Defining the bandwidth of the transmitted signal $x(t)$ requires some mathematical subtleties. From the perspective of measuring the power spectrum of a signal, the following definition of power spectrum for the transmitted signal is sufficient:

$$P_x(f) \equiv E_x |G_{\text{tx}}(f)|^2 \quad (2)$$

where it is assumed that the transmit constellation is normalized to unit energy. It is clear that $|G_{\text{tx}}(f)|^2$ determines the bandwidth of $x(t)$, where $G_{\text{tx}}(f)$ is the frequency response of the pulse-shape $g_{\text{tx}}(t)$. By normalizing the transmit pulse shape, $\int_f P_x(f)df = E_x$ where E_x is known as the symbol energy and E_x/T as the power.

A simple choice for $g_{\text{tx}}(t)$ is the rectangular pulse-shape. Unfortunately, such a pulse-shape would have poor spectral properties, since the Fourier transform of a rectangular function is a sinc function. This means that a lot of bandwidth would be required to transmit the signal.

Another choice for $g_{\text{tx}}(t)$ is the sinc function. A sinc pulse has ideal spectral properties. For example, if $g_{\text{tx}}(t) = \frac{\sin(\pi t/T)}{\pi t/T}$ then $G_{\text{tx}}(f) = T$ for $f \in [-1/2T, 1/2T]$ giving an absolute bandwidth of $1/2T$. This is the smallest bandwidth a complex pulse amplitude modulated signal could have with symbol rate $1/T$. Unfortunately the sinc pulse shaping filter has a number of problems in practice. Ideal implementations of $g(t)$ do not exist in analog. Digital implementations require truncating the pulse shape, which is a problem since it decays in time with $1/t$. Further the sinc function is sensitive

to sampling errors (not sampling at exactly the right point). For these reasons it is of interest to consider pulse shapes that have excess bandwidth, which means the double sided bandwidth of the pulse-shape is greater than $1/T$.

1.1 A Common Pulse Shape

The most common Nyquist pulse, besides the sinc pulse, is the raised cosine. The raised cosine pulse-shape has Fourier spectrum

$$G_{\text{rc}}(f) = \begin{cases} T, & 0 \leq |f| \leq (1-\alpha)/2T \\ \frac{T}{2} \left[1 + \cos \frac{\pi T}{\alpha} \left(|f| - \frac{1-\alpha}{2T} \right) \right], & \frac{1-\alpha}{2T} \leq |f| \leq \frac{1+\alpha}{2T} \\ 0, & |f| > \frac{1+\alpha}{2T} \end{cases}$$

and transform

$$g_{\text{rc}} = \frac{\sin \pi t/T}{\pi t/T} \frac{\cos(\pi \alpha t)}{1 - 4\alpha^2 t^2/T^2}.$$

The parameter α is the rolloff factor, $0 \leq \alpha \leq 1$ (sometimes β is used instead of α) and is often expressed as percentage excess bandwidth. In other words, 50% excess bandwidth would correspond to $\alpha = 0.5$. At large values of α the tails of the time-domain pulse decay quickly, but the spectrum roll off is smoother (i.e., it requires more excess bandwidth). Reducing the time-domain tails of the pulse shape reduces the amount of ISI introduced by sampling error. Thus rolloff factor α can be used to tradeoff excess bandwidth for resistance to ISI caused by sampling errors. The raised cosine pulse is illustrated in Figure 2.

The transmitter does not use the raised cosine pulse directly. The reason is that the raised cosine pulse shape corresponds to $g(t)$ while we use $g_{\text{tx}}(t)$ at the transmitter and $g_{\text{rx}}(t)$ at the receiver. Recall the $g(t) = \int g_{\text{rx}}(\tau) g_{\text{tx}}(t - \tau) d\tau$ and that $g_{\text{rx}}(t) = g_{\text{tx}}(-t)$. In the frequency domain this means that $G(f) = G_{\text{tx}}(f) G_{\text{rx}}^*(f)$. Consequently we choose $g_{\text{tx}}(t)$ to be a “square root” of the raised cosine. Such a pulse shape is known as a square root raised cosine, or a root raised cosine and is written

$$g_{\text{sqr}}(t) = \frac{4\alpha}{\pi \sqrt{T}} \frac{\cos[(1+\alpha)\pi t/T] + \frac{\sin[(1-\alpha)\pi t/T]}{4\alpha t/T}}{1 - (4\alpha t/T)^2}.$$

Of particular interest note that $g_{\text{sqr}}(t)$ is even, thus if $g_{\text{tx}}(t) = g_{\text{sqr}}(t)$ then $g_{\text{rx}}(t) = g_{\text{tx}}(-t) = g_{\text{tx}}(t)$ and the transmit pulse shape and receive pulse shape are identical. The square root raised cosine is its own matched filter!

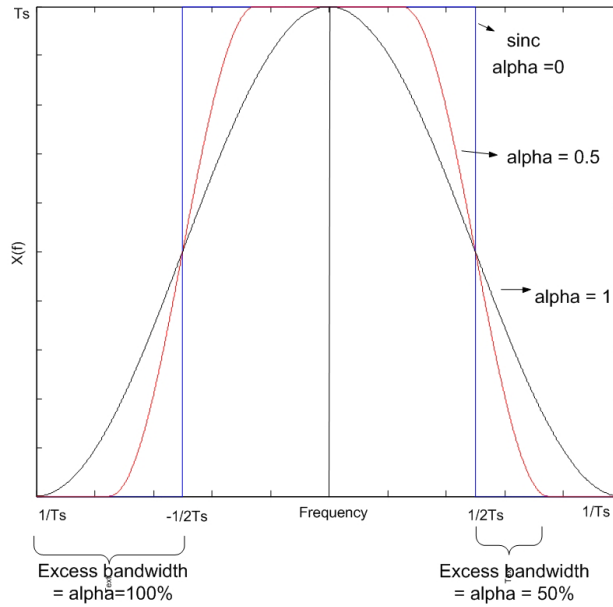


Figure 2: Spectral response of the raised cosine pulse shaping function $g(t)$.

1.2 Practical Implementation of Pulse Shaping

Recall that the Nyquist theorem states it is sufficient to sample a given signal at a sampling frequency is at least twice the largest frequency in the signal, to be able to reconstruct the signal effectively. Sampling at a rate higher than what is required by the Nyquist theorem is known as oversampling. Oversampling at the analog front ends of a digital communication system (DAC and ADC) will make the design of all the filters much simpler. The increased bandwidth, due to a larger sampling frequency, will relax the sharp cutoff requirements on the filters needed to remove aliasing effects. Oversampling in digital systems is also important for efficient synchronization.

The upsampling and downsampling processes are critical in order to reap the benefits of oversampling. Upsampling a sequence $\{s[n]\}$ by a factor L is equivalent to inserting $L - 1$ zeros between every sample of the sequence. Downsampling a sequence $\{y[n]\}$ by a factor M is equivalent to throwing away $M - 1$ out of every M symbols (i.e., $y_{\text{downsamp}}[k] = y[kM]$). Upsampling and downsampling are used to facilitate the digital implementation of $g_{\text{tx}}(t)$ and $g_{\text{rx}}(t)$.

Simple explanations of transmit and receive pulse shaping using the concepts of upsampling and downsampling are provided in this lab. More efficient implementation computationally is possible using filter banks but that is not required to complete the assignment.

Transmit Pulse Shaping

The challenge with implementing transmit pulse shaping is that the symbol rate may be less than the sample rate required for pulse shapes with excess bandwidth. For example, for a square root raised cosine pulse shape, the Nyquist rate is $(1 + \alpha)/T$ while the symbol rate is only $1/T$.

Let T_x be some sampling period such that $1/T_x$ is greater than twice the maximum frequency of $g_{tx}(t)$. For simplicity we take $T_x = T/L$. Other choices of T_x would require a resampling operation. The continuous-time complex baseband signal is

$$x(t) = \sqrt{E_x} \sum_{n=-\infty}^{\infty} s[n] g_{tx}(t - nT). \quad (3)$$

Since $x(t)$ is bandlimited by virtue of the bandlimited pulse shape $g_{tx}(t)$, there exists a sequence $\{c[n]\}$ such that

$$x(t) = \sqrt{E_x} \sum_{n=-\infty}^{\infty} c[n] \frac{\sin((t - nT_x)/T_x)}{(t - nT_x)/T_x},$$

where $c[n] = x(nT_x)$. Substituting Eq. (3) gives

$$\begin{aligned} c[n] &= \sum_{m=-\infty}^{\infty} s[m] g_{tx}(nT_x - mT) \\ &= \sum_{m=-\infty}^{\infty} s[m] g_{tx}[n - mL], \end{aligned} \quad (4)$$

where $g_{tx}[n] = g_{tx}(nT_x)$. Using multi-rate signal processing identities, it can be shown that Eq. (4) is the result of upsampling $s[m]$ by L , then filtering with $g_{tx}[n]$.

Receiver Pulse Shaping

It is easier to justify the digital implementation of receiver pulse shaping. Let $z(t)$ denote the complex baseband input to the continuous-to-discrete converter. Assume that $z(t)$ has already been bandlimited by the RF in the

analog front end. Let $T_z = T/M$ for some integer M such that $1/T_z$ is greater than the Nyquist rate of the signal. This is known as *oversampling*. Let $z[n] := z(nT_z)$. Using results on filtering bandlimited signals, it can be shown that

$$y[n] = \int g_{rx}(\tau) z(nT - \tau) d\tau = T_z \sum_{m=-\infty}^{\infty} z[m] g_{rx}((nM - m)T_z) \quad (5)$$

$$= \sum_{m=-\infty}^{\infty} z[m] g_{rx}[nM - m], \quad (6)$$

where $g_{rx}[n] = T_z g_{rx}(nT_z)$. Using multi-rate signal processing identities, it can be shown that Eq. (6) is the result of downsampling the filtered oversampled signal $z[n]$ by $g_{rx}[n]$.

1.3 Pulse Shaping in LabVIEW

LabVIEW provides many useful digital signal processing tools. In this lab, you will use *MT Generate Filter Coefficients.vi*, a Modulation Toolkit VI found in the Modulation palette (**Function**⇒**Addons**⇒**Modulation**⇒**Digital**⇒**Utilities**⇒**MT Generate Filter Coefficients.vi**). This Modulation Toolkit VI allows you to generate a pulse shaping filter (e.g., root raised cosine) and its associated matched filter based on a set of input parameters. You will need to use this VI to build the pulse shaping and matched filtering blocks of Figure 1. The inputs to this VI [*modulation type*, *pulse shaping filter*, *filter parameter* and *filter length (symbols)*] can be obtained from the *pulse shaping parameters* available in the *modulation parameters* cluster.

As a note about code style in LabVIEW, when building a VI requiring many inputs/outputs (i.e., about five or more of each), you should bundle related inputs/outputs into clusters. Clusters, found in the Cluster palette, allow you to group together terminals of different datatypes. Please refer to [12] for more information on how to build a cluster or access the elements of a cluster. For an example of a cluster you will be using in the lab, please look at the *pulse shaping parameters* terminal, which can be accessed from the front panel of *top.tx.vi*.

2 Pre-Lab

To prepare for this part of the lab you will be required to build the pulse shaping and matched filtering blocks of Figure 1. Tables 1 and 2 describe the details of the VIs you must implement. You will plug your code into *receiver.vi* and *transmitter.vi* and run the simulator provided to you in order

Table 1: Description of *pulse_shaping.vi*

<i>pulse_shaping.vi</i> - oversample input and apply transmit pulse shaping filter			
INPUTS	<i>input</i>	array of CDB (complex doubles)	input symbol sequence
OUTPUTS	<i>output</i>	array of CDB	oversampled input convolved with pulse shaping filter

Table 2: Description of *matched_filtering.vi*

<i>matched_filtering.vi</i> - apply matched filter corresponding to pulse shaping parameters			
INPUTS	<i>input complex waveform</i>	IQ Waveform cluster ¹	received sequence
OUTPUTS	<i>output complex waveform</i>	IQ Waveform cluster	received sequence convolved with matched filter

to verify functionality and correctness. Please name your VIs as per the convention *yourname.Viname.vi*.

You have been provided with templates for the VIs you need to create for this lab that already have all the inputs and outputs wired for you. What is required of you is to finish constructing the block diagram to provide the functionality of the VIs.

Throughout the course, several of the VIs you will create will have a *modulation parameters* cluster passed into and out of them. The *modulation parameters in* contains many of the parameters needed by your VIs and will be unbundled from them. This has already been done for you in the template VIs if necessary. Some VIs will also have *modulation parameters out*, so that the cluster can be passed on through to the VIs that follow. Please ensure that these clusters remain wired the way they are in the template VIs, as changing how they are wired will cause VIs further down the line to break.

Figure 3 shows the dependencies between the files that make up the simulator. The top level of this simulator, *simulator.vi*, connects *top_tx.vi* to

¹IQ Waveform clusters contain three elements: *t0*, *dt*, and *Y*. Please recall from Part 2 of Lab 1 what these elements are and what their data types are.

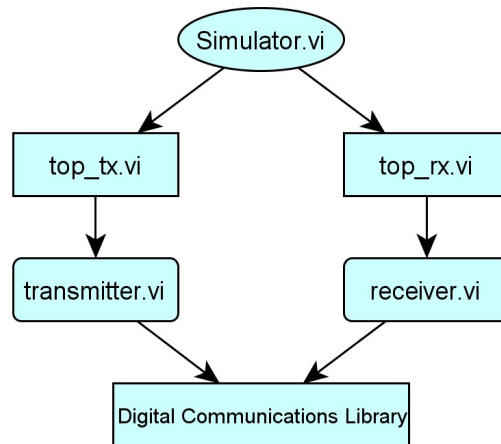


Figure 3: Hierarchy of code framework for new simulator.

top_rx.vi and provides each with the appropriate inputs. The parts of the simulator you will be modifying are located in *transmitter.vi* and *receiver.vi* shown in Figures 4 and 5 respectively.

You will be putting your VIs into *transmitter.vi* and *receiver.vi*, replacing the locked versions that are already there. After doing this, you will then open up *simulator.vi*, that you will use to confirm your VIs operate correctly before implementing them on the NI-USRP.

Notice that *pulse_shaping.vi* and *matched_filtering.vi* do not take any parameters as inputs. All of the pulse shaping and oversampling parameters you need to use for these VIs can be accessed from the *modulation parameters* in cluster. After building these VIs, replace the existing code in the simulator with your code. Replace a subVI in the transmitter or receiver with your code

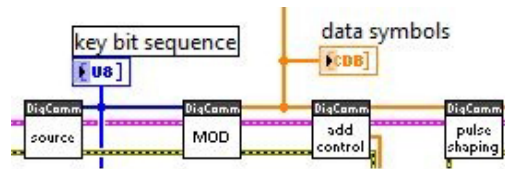


Figure 4: Block diagram of *transmitter.vi*.

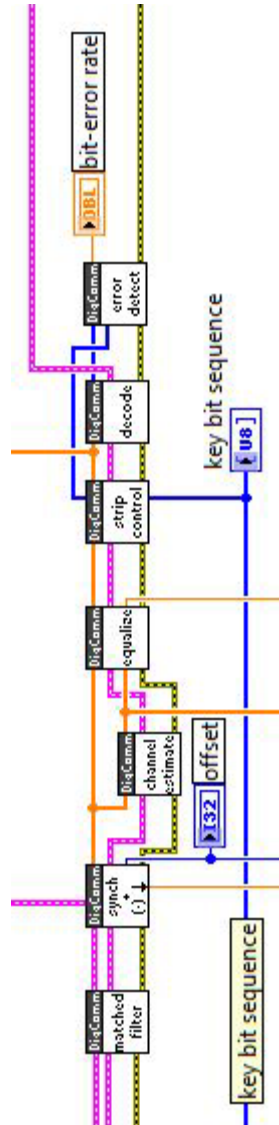


Figure 5: Block diagram of receiver.

by right-clicking on the appropriate VI and selecting **Replace⇒Select a VI**. Remember to wire error clusters where appropriate.

As described in the previous section, use *MT Generate Filter Coefficients.vi* from the Modulation Toolkit to generate your pulse shaping and matched filters. After generating each filter, you can apply the filter by convolving it with the complex array component of the appropriate input waveform. Do not use a Modulation Toolkit VI to apply either the pulse shaping filter or matched filter.

In LabVIEW, *Convolution.vi*, which is located in the Signal Processing tool palette (**Functions⇒Signal Processing⇒Signal Operation⇒Convolution.vi**), supports complex convolution. You can use this VI when creating your subVIs for this lab and others.

In summary:

- Use *MT Generate Filter Coefficients.vi* to generate the pulse shaping filter and its associated matched filters. Do not use Modulation Toolkit VIs to apply these filters. Instead, use convolution, and remember to make any necessary changes needed for complex convolution.
- The *modulation parameters* cluster contains many of the parameters needed for pulse shaping, including *pulse shaping parameters*, *TX oversample factor*, and *RX oversample factor*.
- After building *pulse_shaping.vi* and *matched_filtering.vi*, insert them into the simulator provided to you to verify functionality and correctness.

Using an Eye Diagram as an Analytical Tool

In this lab you will use an eye diagram to make some qualitative statements about the received signal. An eye diagram is constructed by taking a time domain signal and overlapping traces of a certain number of symbol times. For example, if $T_s = 10 \mu\text{sec}$ is the symbol time for a given waveform, an eye diagram for an eye length of two would be generated by overlaying traces of duration $2T_s$ starting at an ideal sampling point. In the absence of noise, these overlapped signals might look something like Figure 6.

The eye diagram of Figure 6 shows the overlapped traces of a BPSK modulated bit stream using a sinc pulse for an eye length of two. This eye diagram can give us some qualitative insight about our received signal. For example, the opening of the eye indicates the period of time over which we can successfully sample the waveform (i.e., without suffering from ISI). Along with indicating the amount of ISI in the system, the distortion of the signal in the diagram also indicates various qualitative parameters, such as